DAY 07 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

ERC-20 tokens

Wednesday 21 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-07

Learning objectives

1. Explain what a token really is (a balance table in a contract) and what ERC-20 promises.
2. Use decimals correctly: the chain stores whole numbers, wallets add the dot.
3. Understand allowances: `approve`, `transferFrom`, the unlimited approval, and the approve race.
4. Use `permit` (EIP-2612) to approve with a signature, and survive its front-running.
5. Build a vault for any token that copes with fee-on-transfer, no-return, false-return and callback tokens.
6. Test a standard, not a contract: one behaviour suite, differential testing and model-based random tests.

The concept, step by step

Step 1 — A token is a table inside a contract

There are no coins to hold. "Ram owns 5 tokens" only means `balanceOf[Ram] == 5 × 10¹⁸`. ERC-20 is a promise about function names and two events, so every wallet and exchange can talk to every token the same way. In `MiniToken` every change goes through one function, `_update(from, to, value)`: `from == 0` is a mint, `to == 0` a burn.

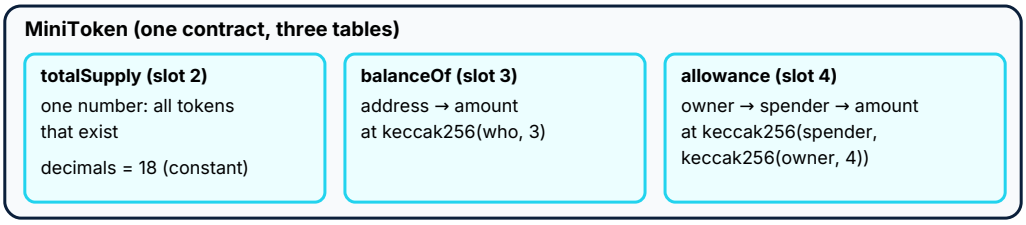


Figure 1 — Where MiniToken keeps its data. The tests open these slots with `eth_getStorageAt` (Day 05).

Step 2 — Decimals are a display trick

The EVM has no fractions. `decimals = 18` tells a wallet to show `balance ÷ 10¹⁸`. 2.5 tokens is 2,500,000,000,000,000,000 units at 18 decimals and 2,500,000 at USDC's 6. Dividing 1 token by 3 gives 333,333,333,333,333,333 each and one unit left over: integer division rounds down.

Step 3 — Letting someone else spend your tokens

`approve(spender, 100)` only records a permission; nothing moves. Later the spender calls `transferFrom(owner, to, 40)`: the token checks the allowance, lowers it to 60, and moves the tokens. The allowance is re-checked at every use against the balance *at that moment*. An allowance of `2²⁵⁶ − 1` means "unlimited" and is never reduced (saves a storage write, but exposes everything you will ever hold).

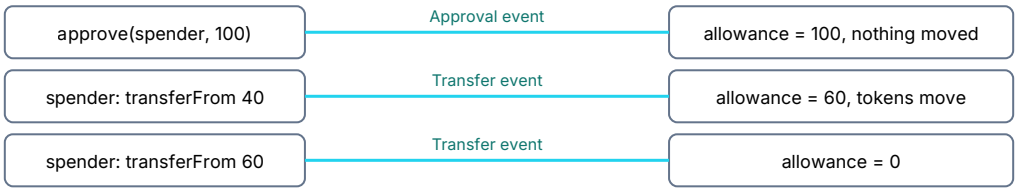


Figure 2 — Spending an allowance: the token lowers it as it goes; a third spend would revert.

Step 4 — The approve race

Alice has approved Mallory for 100 and now sends `approve(50)` to lower it. Mallory sees it pending and front-runs with `transferFrom(100)`; then Alice's `approve(50)` lands and Mallory spends the *new* 50. Total taken: 150, though Alice only ever meant 100 or 50 (measured: 150 units). Fixes: `increase/decreaseAllowance`, a compare-and-set `approveIfCurrent`, or `permit`.

Step 5 — Permit: approve with a signature

EIP-2612: the owner signs a typed message (the Day 03 kind of signature) naming spender, value, a **nonce** and a **deadline**. Anyone can submit it to `permit`; the token recovers the signer, sets the allowance and bumps the nonce, so the signature works once. The owner sends no transaction and pays no gas. The catch: once broadcast, the signature is public, so anyone can use it first.

Key code walkthrough

The one place balances change (`contracts/MiniToken.sol`)

```
function _update(address from, address to, uint256 value) internal virtual {
    if (from == address(0)) totalSupply += value;                    // mint
    else {
        uint256 fromBalance = balanceOf[from];
        if (fromBalance < value) revert ERC20InsufficientBalance(from, fromBalance, value);
        unchecked { balanceOf[from] = fromBalance - value; }         // checked just above
    }
    if (to == address(0)) unchecked { totalSupply -= value; }        // burn
    else unchecked { balanceOf[to] += value; }                       // sum of balances ==
totalSupply
    emit Transfer(from, to, value);
}
```

Credit what ARRIVED (`contracts/TokenVault.sol`)

```
uint256 before = token.balanceOf(address(this));
token.safeTransferFrom(msg.sender, address(this), amount);   // handles no-return and false-
return tokens
received = token.balanceOf(address(this)) - before;          // fee tokens deliver less than
`amount`
if (received == 0) revert NothingReceived();                 // a token can "succeed" and move
nothing
balances[token][msg.sender] += received;
```

Permit that cannot be griefed

```
try IERC20Permit(address(token)).permit(msg.sender, address(this), amount, deadline, v, r, s) {}
catch {}
return _deposit(token, amount);   // if the allowance is really missing, transferFrom reverts
clearly
```

Results from the run

Run	Result
Gas: transfer	To a new holder 51,543; to an existing holder 34,443 (a new slot costs ~17,000+ more). Hand-written token 34,443 vs OpenZeppelin 34,460
Gas: allowance	transferFrom with a finite allowance 40,488; with an unlimited one 37,249
Approve race	Spender ends with 150 units after "100, then lower to 50"
Fee token (1 %)	Naive vault: credits 2000, holds 1980, the last user cannot withdraw. TokenVault: credits 1980, solvent
Weird tokens	USDT-style (no return): TokenVault deposits via SafeERC20. Returns false: `SafeERC20FailedOperation`. Ghost token: `NothingReceived`
Hook token	Re-entry via withdraw, deposit and depositWithPermit refused; inside the callback the vault already shows balance 0, total 0
Quality	167 / 167 tests · ESLint + Solhint clean · `npm audit --omit=dev` 0 · Slither: no real issue · mutation testing 34 / 34 caught

How I tested it

1. One behaviour suite, three tokens

31 checks describe ERC-20 itself (events, boundaries, error data, allowances). The same file runs against `MiniToken`, plain OpenZeppelin and `LearningToken`: if all three pass, the checks describe the standard, not one contract.

2. Differential testing against OpenZeppelin

150 random programs run on MiniToken and OpenZeppelin side by side. After every operation they must agree on success, error name and arguments, events, every balance and allowance, and `totalSupply`. OpenZeppelin is the judge of my token.

3. Random operations against a model

200 random programs on `LearningToken` versus a JavaScript model of a capped ERC-20: sum of balances equals supply, supply never passes the cap, a failed call changes nothing.

4. Controls, then mutants

Every "the vault survives X" has a naive vault that fails X. Then `npm run qa:mutation` breaks the contracts 34 ways. First pass: 31 caught, 3 survived (an untested `NothingReceived` and two untested reentrancy guards). After adding tests: 34 / 34.

QA & security findings

I wrote code and tests in the morning and attacked both in the evening, so the findings are things I designed for but had not proven.

ID	Sev.	Problem	Fix
T1	Low	`NothingReceived` was never exercised: removing the check changed no test result.	`GhostToken` (returns true, moves nothing) plus a naive-vault control.
T2	Low	Reentrancy guards on `deposit` and `depositWithPermit` untested: the attacker only re-entered `withdraw`.	Attacker modes that re-enter each entry point from the token callback.
I1	Info	Depositing "a token" that is an ordinary account was untested (it reverts).	Test added.

Slither: 3 results in production code, all in `_deposit`: the balance-before/after pattern must write state after the external call, protected by `nonReentrant`; the `== 0` check is intended. **Accepted:** the approve race is inherent to ERC-20; rebasing tokens are unsupported; the owner can mint up to the cap; not audited. Do not deploy with real funds.

Common mistakes

- Crediting the `amount` a user asked to deposit instead of what arrived (fee-on-transfer tokens).
- Calling `token.transfer(...)` through a plain `IERC20` on USDT: the empty return value makes the call revert. Use `SafeERC20`.
- Treating `approve` as additive, or lowering an allowance while the spender may be spending it.
- Giving unlimited approvals to contracts you do not fully trust.
- Calling `permit` without a fallback: a front-run signature makes the whole transaction revert.
- Assuming `permit` worked: some tokens have a fallback that "succeeds" without setting anything.
- Forgetting that a token can call you back (ERC-777 style hooks) in the middle of your function.

Interview questions

1. What does `approve` do, and why does it overwrite?

It records how much a spender may still move. It sets, not adds, so the allowance is exactly what you last said; the cost is the race when you change a non-zero value.

2. Why must a vault measure the balance before and after?

Because a token may deliver less than requested (fees) or nothing at all while reporting success. Only the balance difference says what arrived.

3. What is wrong with `token.transfer(...)` on USDT?

USDT returns nothing, but `IERC20` expects a `bool`; decoding empty data reverts. `SafeERC20` accepts "no data" as success and "false" as failure.

4. How can `permit` be front-run, and what is the defence?

The signature is public once broadcast, so someone submits it first and uses up the nonce. Wrap `permit` in `try/catch` and let the later `transferFrom` decide.

5. What is differential testing?

Running two implementations of the same specification on identical random inputs and requiring identical results; any difference is a bug in one of them.

Exercises summary

Level	Task	Result of the solution
Easy	increase/decreaseAllowance, burnFrom, explain decimals	100 → 150 → 120; decrease(999) reverts; 2.5 tokens = 2.5×10^{18} units
Medium	Capped owner-only mint, approve race and `approveIfCurrent`, supply invariant	Mint stops at the cap; compare-and-set reverts after the front-run
Challenge	Naive vault vs fee token, permit front-running, differential test	Naive: credited 2000, holds 1980; TokenVault solvent; deposit survives the front-run

Cheat sheet

`transfer(to, v)`	move your tokens · `Transfer(from, to, v)`
`approve(s, v)`	set allowance (overwrites) · `Approval(owner, s, v)`
`transferFrom(f, t, v)`	needs allowance[f][caller] ≥ v; lowers it (unless $2^{256} - 1$)
mapping slots	`balanceOf` at keccak256(who, 3) · `allowance` at keccak256(spender, keccak256(owner, 4))
commands	`npm start · test · lint · slither · qa:mutation · ex:easy|medium|challenge`

Key terms

Term	Plain meaning
ERC-20	The standard functions and events of a fungible token
Base unit	The smallest indivisible amount: 1 token = 10^{decimals} units
Allowance	How much a spender may still move from an owner
Approve race	Changing an allowance as it is being spent can let a spender take both amounts
Permit	Approve with a signed message; the nonce makes it one-time
Fee-on-transfer	A token whose receiver gets less than the sender sent
SafeERC20	A wrapper that handles tokens returning nothing or false
Differential testing	Two implementations, identical inputs, identical results required

What I'd explore next

- Day 08: ERC-721 and ERC-1155, safe transfers and receiver hooks, royalties.
- Day 09: front-running and access-control exploits in depth; timelocks and multisig owners.
- Later: ERC-4626 vaults (share accounting), and the inflation attack on them.

References

1. EIP-20: Token Standard. https://eips.ethereum.org/EIPS/eip-20
2. EIP-2612: Permit extension for EIP-20. https://eips.ethereum.org/EIPS/eip-2612 · EIP-712. https://eips.ethereum.org/EIPS/eip-712
3. ERC-6093: Standard custom errors for tokens. https://eips.ethereum.org/EIPS/eip-6093
4. OpenZeppelin Contracts 5.x: ERC20 and SafeERC20. https://docs.openzeppelin.com/contracts/5.x/erc20
5. SWC-114: Transaction order dependence. https://swcregistry.io/docs/SWC-114
6. Solidity docs 0.8.24: Security considerations. https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
7. fast-check: property-based testing. https://fast-check.dev/ · Slither. https://github.com/crytic/slither